

第4回

手嶋毅志 (Takeshi Teshima)

質問と回答

加算対象ではないその他の質問・コメント

ビッグデータの普及により、帰納バイアスや事前知識の重要性は今後どのように変化するとお考えですか？

- ①ビッグデータがある分野では、効率よく学習するための工夫という役割になっていきます.
- ②ビッグデータが相対的に生まれづらい領域もありますね（医療や人道分野など）.

統計的機械学習とディープラーニングの違いは何でしょうか？

第6回あたりから出てきますが、ディープラーニング（深層学習）は、深層ニューラルネットという種類のモデルを用いて（統計的）機械学習を行うことを指します.

加点対象ではないその他の質問・コメント

過適合について、リスクを最小化するために、パラメーターの大きさの罰則項をつけることがあり、正則化係数 λ によってその強さを調整していると学びました。罰則項を追加することによって、悪影響が出るケースはあるのでしょうか？

正則化係数やハイパーパラメーターを決める際に、1通りの値で実験を行うと偏った結果が出る、ということにはならないのでしょうか。パラメーターの値を変更して実験を行うことはありますか。それとも、検証曲線のように、先に最適な λ の予測を立てるので、それだけを使って評価をすることが一般的なののでしょうか。

例えば正則化係数 λ が大きすぎる場合などは、まさしく悪影響があります。

それを避けるために、 λ （や他のハイパーパラメーター）の候補を色々試して（＝パラメーターの値を変更して）その中で良さそうだったものを使います。その選択をデータに基づいて行うのが、検証データの取り置き法などによるモデル選択です。

加点对象の質問

学習の枠組みに関する講義がありましたが、教師あり学習と教師なし学習では、学習戦略はどのように異なるのでしょうか？

教師なし学習とは「ラベルなしデータ（特徴量のみ）だけを集めて学習する」という学習戦略のことです。教師付き学習との違いは、ラベルデータの有無です。

※予測タスクなら教師付き学習，と結びつけてしまいがち（確かにそれが基本）ではありますが，教師無し学習で予測タスクを解くという試みも存在します。

質問（続き）

データの集め方によって，学習戦略はいくつかに大別されます：

教師付き学習，教師無し学習，半教師付き学習，能動学習，強化学習，……

- 「教師付き学習」（supervised learning）：ラベル付きデータだけを集めて学習
- 「教師無し学習」（unsupervised learning）：ラベル無しデータだけを集めて学習
- 「半教師付き学習」（semi-supervised learning）：少数のラベル付きデータと，多数のラベル無しデータを集めて学習

などなど

【用語】これらは，どのようなデータの集め方で学習するかという話であって，目的とするタスク設定とは別の話であることに注意。

加対象の質問

モデルの選択基準において、データ量が多ければ置き法で済ませることが多いとありましたが、置き法がLOOCV等と比較して優れている点は何ですか。

置き法がLOOCV等と比較して優れている点は、**計算時間**です。

1組のハイパーパラメーターを評価するために、LOOCVでは n 回の学習～評価を実行しますが、置き法では1回の学習～評価だけで済みます。

学習時間は、前回までに出てきた手法では解の公式がありほぼ無視できましたが、モデルやデータ量次第では、**1回の学習をするだけで数時間に及ぶ場合**もよくあります。

質問（続き）

（小ネタ的な余談）

LOOCVでは、「普通は」1組のハイパーパラメーターを評価するために、 n 回の学習～評価を実行することになります。

しかし、線型モデルの最小二乗回帰という超特殊なケースにおいては、LOOCVの評価値を一発で算出できる次の式が知られています。

$$H := \Phi(\Phi^T\Phi)^{-1}\Phi^T \text{ の対角成分を } \{H_{i,i}\}_{i=1}^n \text{ として, } \frac{1}{n} \sum_{i=1}^n \left(\frac{y_i - \hat{\theta}^T \phi(x_i)}{1 - H_{i,i}} \right)^2.$$

【補足】 ここで、 $\hat{\theta}$ は全データを使ったときの最適なパラメーター $\hat{\theta} := (\Phi^T\Phi)^{-1}\Phi^T\mathbf{y}$ 。仕組みは次のとおり：パラメーターの解の公式を詳しく調べると、検証用の1点を除いたときの解 $\hat{\theta}_{-i}$ を、全ての点を用いたときの解 $\hat{\theta}$ を用いて表せる → 検証用の1点への予測値も明示的に書ける → その点での検証誤差も書き下せる → その平均であるLOOCVの評価値も書き下せる。

【参考】 式 (12.23) of Seber, G. A. F., & Lee, A. J. (2003). Linear Regression Analysis (2nd ed.). Wiley.

加対象の質問

ハイパーパラメーターを選択するにあたって、ランダムサーチが手軽（＝ベイズ最適化が手軽でない？）理由が知りたいです。既に得ている目的関数地の情報から当たりをつけながら、逐次的に探索するベイズ最適化を行うことで効率よく探索することができると考えましたが、実際のところ面倒くさいのでしょうか？

ありがとうございます，良い質問ですね．

はい，手軽さだけの比較でいえば，ランダムサーチのほうが，かなり手軽です．

①実装の手軽さ，②追加的な考慮事項の少なさ，という2つの側面を説明します．
（つづく）

質問（続き）

①実装の手軽さ

ランダムサーチは最初に候補となるハイパーパラメーターの組を作成してしまえば、特別な工夫は何も要らず、学習～評価～結果の保存だけすればよいので実装が簡単。さらに、**並列**で実行してもよいので、結果が出揃うまでの実時間を短縮できる。

ベイズ最適化は、既に評価した結果を見て次の候補を作る手法なので、その候補生成の仕組みをプログラムに加える必要がある。それを簡単に実現するためのライブラリ（optunaなどが有名）はあるが、実用的には何かと複雑な実装になってしまいがち。さらに、仕組み上、**基本的には複数の候補を並列で実行できない**（すでに評価済みの結果を使って次の候補を決める、という方法で当たりをつけていくため）。

【補足】なるべく並列実行をできるようにするためのベイズ最適化の手法は研究・開発されているが、それを導入することは更に実装の複雑さが上がっていくことを意味するので、割に合うかどうかは一概には言いがたい。

質問（続き）

②追加的な考慮事項の少なさ

ベイズ最適化では「入力点」となるハイパーパラメーターの組を順次試します。

最初、あなたは何も情報がない状態で、「各ハイパーパラメーターが取れる値の範囲」の情報だけからスタートします。

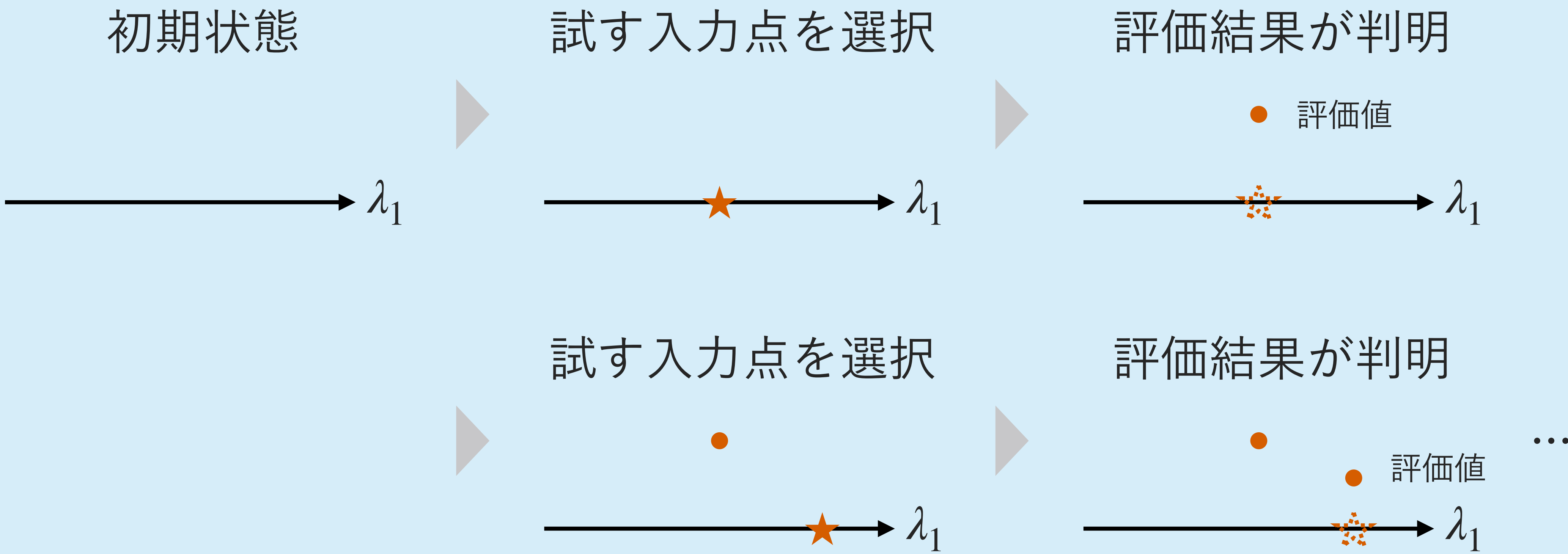
（例： $\lambda_1 \in [-1, 1]$, $\lambda_2 \in \{0, 1, 2\}$, $\lambda_3 \in [10^{-3}, 10^0]$ など）

そして、目的関数 $\text{Criterion}(\lambda)$ が最小になる入力の組 $(\lambda_1, \lambda_2, \lambda_3)$ を見つけよという課題を解いていきます。

$(\lambda_1, \lambda_2, \lambda_3) \rightarrow \text{Criterion}(\lambda) \rightarrow \text{値が出てくる}$

質問（続き）

簡単のためにハイパーパラメーター1個の場合で図示すると、



質問（続き）

この実行では「探索と活用」のトレードオフを考慮する必要がある。

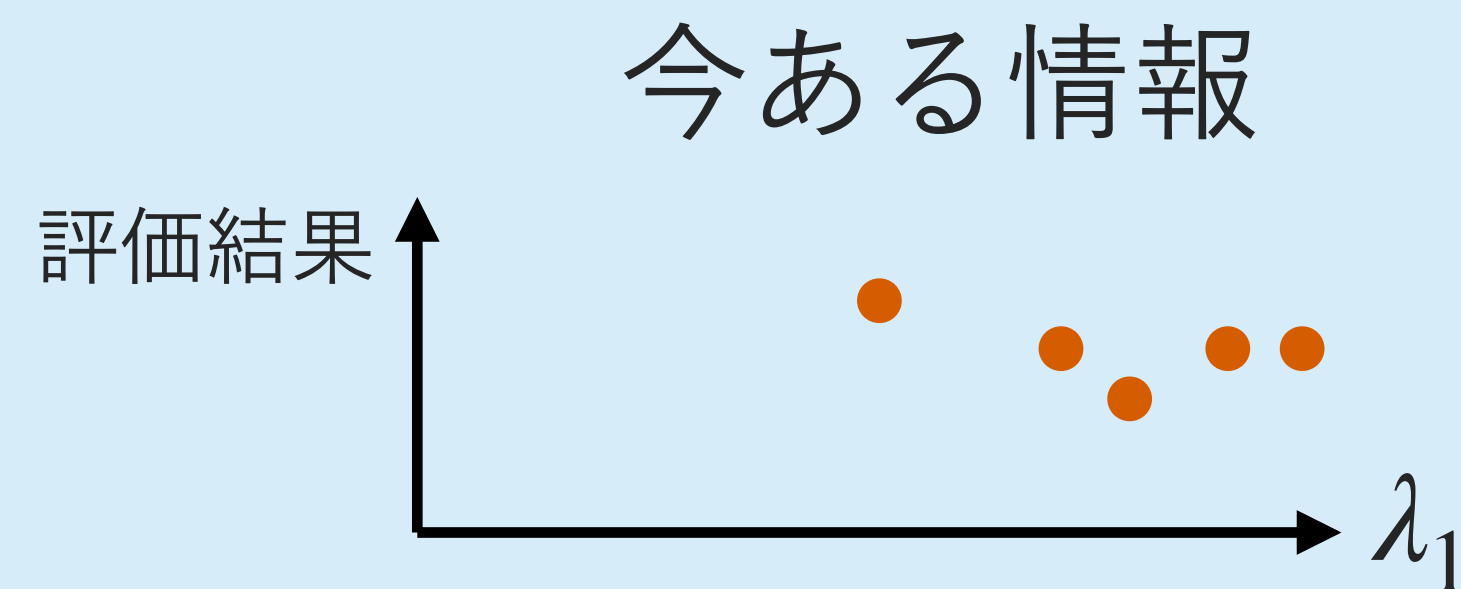
探索（exploration）……情報がまだない辺りの情報収集をしたい

活用（exploitation）……様子がわかってきた辺りで最適解の候補を探したい

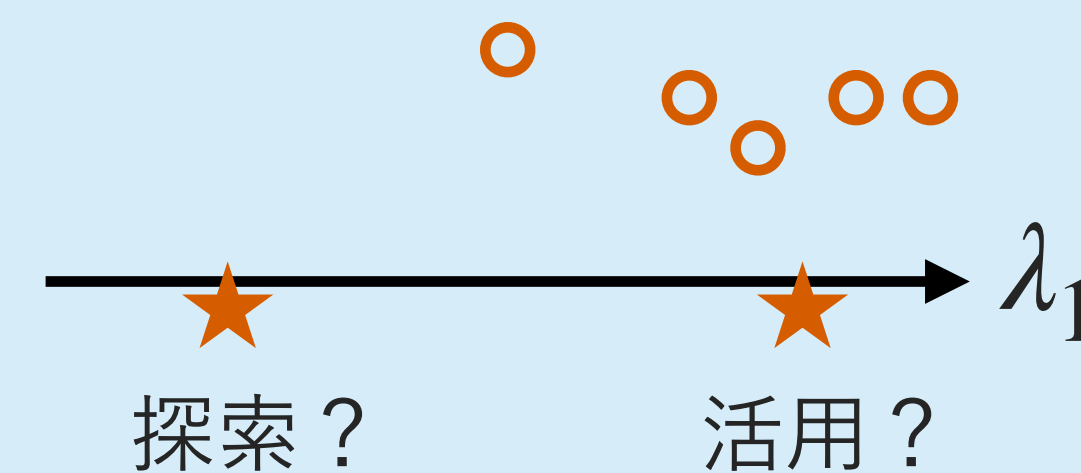
例えば、ハイパーパラメーターを20組試せるぐらいの時間的猶予はあるとする。

20ターン以内に、良いハイパーパラメーターを見つけなければいけない。

次のターン、あなたは探索するか？活用するか？（トレードオフ）



次の候補はどこにする？



質問（続き）

ベイズ最適化の手法では、ふつう、探索と活用のどちらを重視するかを調整できるパラメーターが何かしらあり、それを調節する必要がある。
これは、良くも悪くも工夫の余地が少ないランダムサーチとは対比的。

仮にあまり良いハイパーパラメーターが見つからなかったとして、ベイズ最適化では「あまり良い性能のHPが見つからなかった……探索が足りなかったか？」と考えることになる。

一方、ランダムサーチは探索に全振りしているぶん「ランダムにこれだけ探索して良い値が見つからないぐらい難しい問題らしい」というさっぱりした知見は得られる。
そこから、「ベイズ最適化で賢くやったほうが良いか？」「そもそも無理筋な学習タスクか？」などと次のことを考えやすい。

質問（続き）

では、**ベイズ最適化**はいつ不可欠か？

（もちろん、慣れていれば実装負担も探索活用トレードオフも問題にならず、最初からベイズ最適化を使ってもよいかもしれないし、ある程度ランダムサーチして上手くいかなかった場合の積極的な選択肢としては持っておくと良い）

明確に「ベイズ最適化でなくてはいけない」（あるいは他の似た「賢いハイパーパラメーター選択」手法）場面は、許容可能な評価回数が少ない場面。

許容可能な回数は、金銭的な予算や、時間的な予算によって決まってくる。

例えば近年の大規模言語モデルの1回分の学習には、数ヶ月単位での大きな時間がかかる。お金も1回の学習で数百万円～という単位になる。沢山適当に試す余裕はないので、初めから慎重に当たりをつけながら進めざるを得なくなることがある。

加対象の質問

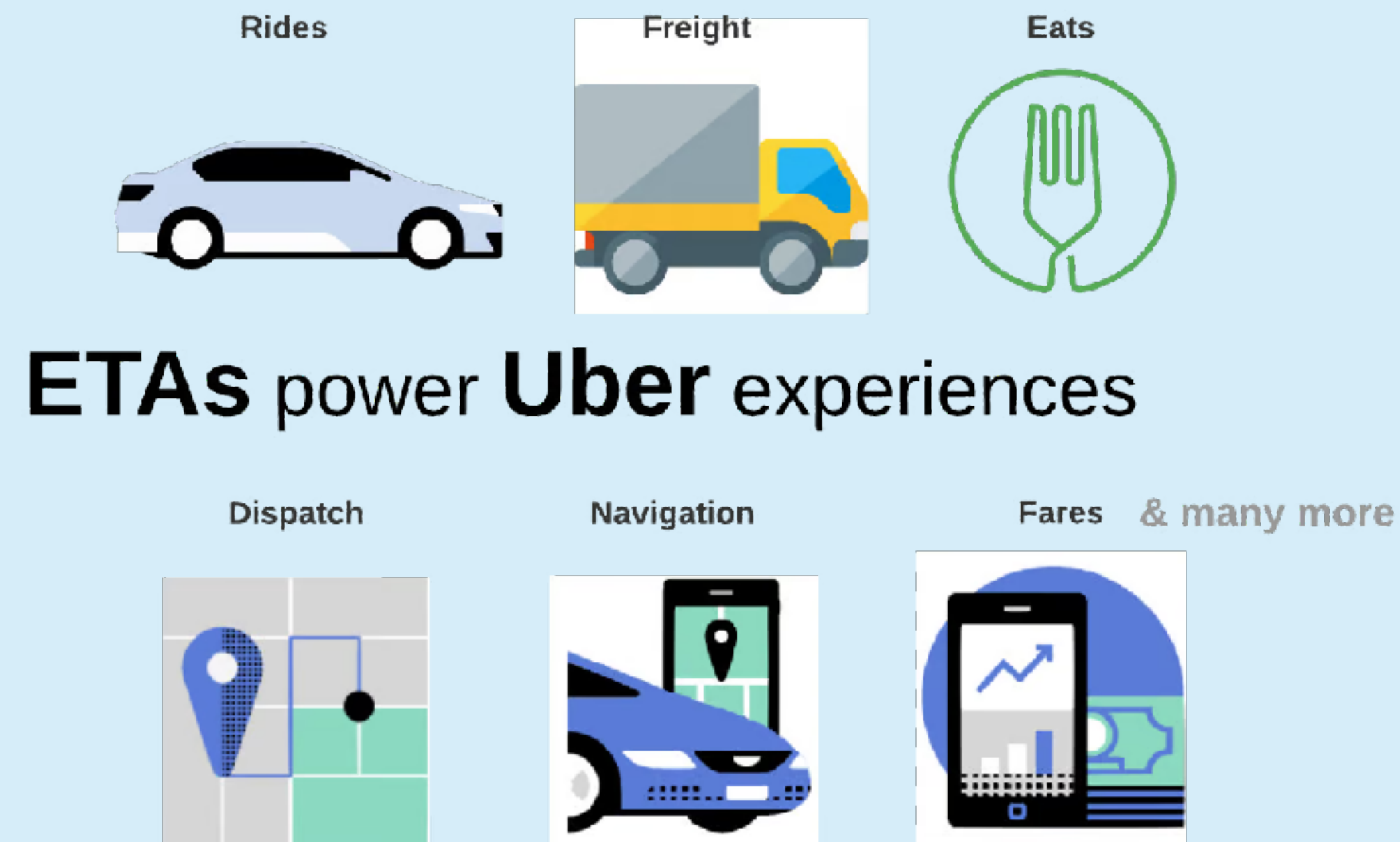
目的関数を設計し、最適化する内容は理解出来たのですが、これらが実務、社会ではどのように活用されているのでしょうか。具体例を教えてください！

ありがとうございます。そろそろ実例の話が欲しい頃ですね。
今回は、既に学んだ回帰タスクの範囲から一つ紹介します。
それは、Uberの到着予定時刻（ETA）予測タスクです。

【出典・参考】 DeepETA: How Uber Predicts Arrival Times Using Deep Learning. (2022, February 10). Uber Blog. <https://www.uber.com/blog/deepeta-how-uber-predicts-arrival-times/>

質問（続き）

到着予定時刻（ETA）が正確に予測できると、お客さんの体験はよくなりますね。



【出典・参考】 DeepETA: How Uber Predicts Arrival Times Using Deep Learning. (2022, February 10). Uber Blog. <https://www.uber.com/blog/deepeta-how-uber-predicts-arrival-times/>

質問（続き）

Uberのテックブログによれば，次の回帰タスクの学習でETA予測が行われています。

特徴量

都市，出発地，目的地，日付，
分単位の時刻，交通状況，
Uber EATSか否か，Uber RIDESか否か，
など

ラベル

（実際の到着時間）から
（経路探索に基づく到着予定時刻）を
引いたもの

※経路探索に基づくETAは，地図データとリアルタイムの交通計測に基づくもので，昔から開発されてきた技術

【出典・参考】 DeepETA: How Uber Predicts Arrival Times Using Deep Learning. (2022, February 10). Uber Blog. <https://www.uber.com/blog/deepeta-how-uber-predicts-arrival-times/>

質問（続き）

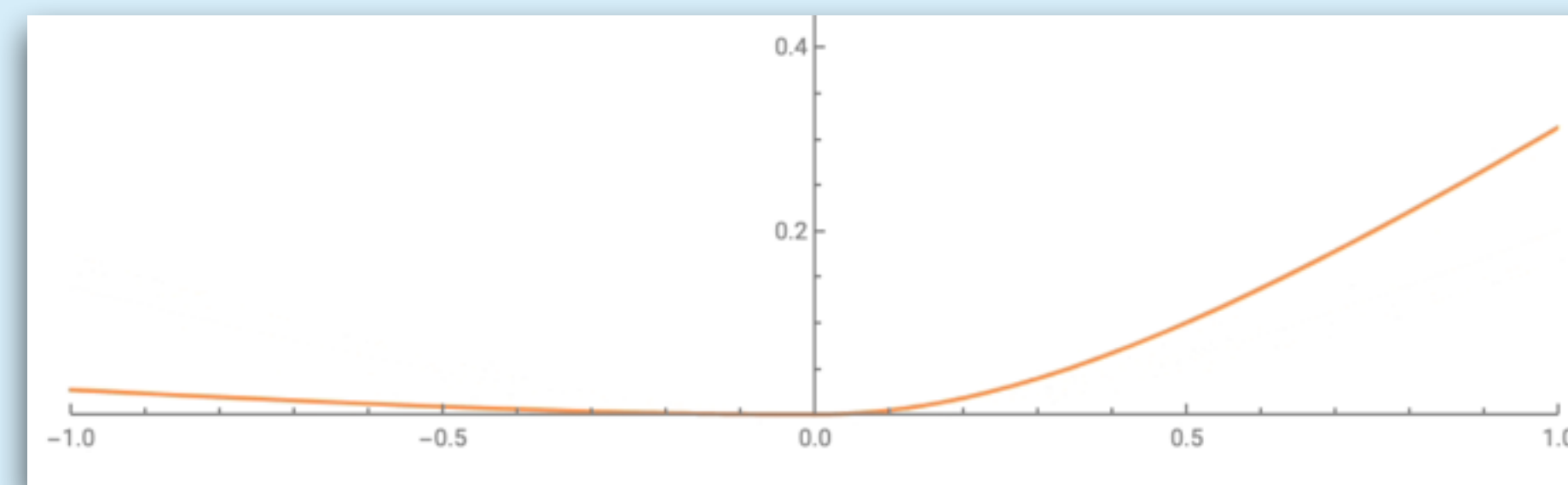
使われている損失関数

Asymmetric Huber Loss（非対称フーバー損失）という名前のもの
（非対称：損失関数のグラフが左右対称ではないということ）

例えば、1分遅れるほうが1分早いよりも圧倒的に悪い，ということを反映.

正解ラベル < 予測値

予測値 < 正解ラベル



【出典・参考】 DeepETA: How Uber Predicts Arrival Times Using Deep Learning. (2022, February 10). Uber Blog. <https://www.uber.com/blog/deepeta-how-uber-predicts-arrival-times/>

質問（続き）

各社が出しているテックブログを調べると、これらの事例は色々出てくるので、ぜひ調べてみてください。

<https://github.com/antoinebri/awesome-ml-blogs?tab=readme-ov-file#big-corps---applied>

論文化されることも多いが、論文だと詳しく書かなくてはいけなくなるので、民間企業内での事例は、各社独自のテックブログのほうが見つけやすいかもしれない。

加点对象の質問

データ量やタスクに応じて帰納バイアスを変化させる手法を取ることのメリット、あるいはデメリットや限界について、伺いたいです。それらについて具体的なアルゴリズム例などがありましたら知りたいです。

良い発想ですね。

「帰納バイアスを変化させる」ことも、ハイパーパラメータ選択の一種と捉えると分かりやすいと思います。検証データでの評価値などによるモデル選択は、「データに基づいて手法の動作を自動的に変える」ことにほかなりません。
では、限界はというと……（つづく）

質問（続き）

限界

「この世にある，ありとあらゆる帰納バイアスの中から，データに基づいて最適なものを選ぶことはできないか？」という試みは，次のような限界に直面します。

手元にあるデータセットだけに基づいて（検証データの取り置き法を使うとして），ハイパーパラメーター候補を無数に試すとどうなるか？

無数のハイパーパラメーター候補に対して評価値を計算すること自体は，時間が許せば可能です。しかし，検証データでの評価は，リスクのサンプル近似にすぎないので，無数のハイパーパラメーター候補を試していくと，「ハイパーパラメーター選択における」過適合をしていってしまいます。

それなので，帰納バイアスの真に完全自動な選択は，実質的に不可能と言ってしまってもよいと思います。

質問（続き）

なお、文字通り「データ量」や「タスクの種類」からハイパーパラメーターを決めるような話も稀にありますが、体系立った整理はされていません。

実際には、データの量やタスクの種類というよりは、データの分布など個別具体的な要因で適切な値が違ふことが多いと思います（思われています）。

「この分野ではこのぐらいの値がいい」という経験知が多少あることもありますが、これも体系立った何かがあるわけではなく、実際には似たデータを扱っている論文をいくつか見つけてきて、その実験のセクションに書いてある値を参考にして、ハイパーパラメーターの候補の探索範囲を決めることなどが多いと思います。

出席確認

本日の出席確認番号は……

2025

告知

- 小テストは月曜日に公開します

ロードマップ

